py -3.13 -m jupyter lab(for my pc)

TechArena 2026 – Phase 1: Click Prediction

Task

Phase 1 is a click-probability prediction task on synthetic recommendation data. No real users or products are involved. Your task is to predict a click probability for every row in `evaluation.csv`.

- An **event** is one recommendation opportunity: one user is shown **10 candidate items** at one moment.
- A **row** in the CSV files is one candidate (an *impression*): one item shown to one user in one event.
- In the training data the target `clicked` is 1 if that candidate was clicked and 0 otherwise.
- An event has **at most one clicked candidate**. Events with no click are also possible: all 10 of their rows have `clicked = 0`.

For **every row of** `evaluation.csv`, predict the probability that the candidate was clicked. Do not submit an extra row or value for the "no click" outcome.

Files

File	Content	Data rows (header excluded)
`train.csv`	Labelled candidates (includes `clicked`), 686,824 events	6,868,240
`evaluation.csv`	Candidates to predict, no labels, 52,939 events (one per user)	529,390
`example_submission.csv`	Format example: one line per evaluation row, all with the same constant value	529,390
`README.txt`	Short description shipped with the data	–

`train.csv` is about 7.3 GB. Consider streaming or chunked processing. Labels for the evaluation rows are not provided.

Columns

`evaluation.csv` has the same columns as `train.csv` except `clicked`.

Column	Meaning
`rowid`	Identifier of the row (one candidate in one event). The key of your submission.
`event_id`	Identifier of the event. All 10 rows of an event share user, timestamp and history.
`user_id`	The user.
`item_id`	The candidate item shown in this row.
`timestamp`	Event time, Unix seconds, UTC (whole minutes).
`time_bucket`	Six-hour block of the UTC day: 0 = 00:00–05:59, 1 = 06:00–11:59, 2 = 12:00–17:59, 3 = 18:00–23:59.
`day_type`	0 = Monday–Friday, 1 = Saturday or Sunday (UTC date).
`history_item_ids`	JSON array of items this user clicked before the event.
`history_timestamps`	JSON array of the Unix-second times of those clicks.
`clicked`	**Training only.** 1 = clicked, 0 = not clicked.

- `rowid`, `event_id`, `user_id` and `item_id` are **opaque identifiers**. Treat them as text, not numbers, and do not try to interpret them. `rowid` and `event_id` are random and carry no information; use them as keys only.
- The history columns hold JSON arrays inside quoted CSV fields. Read the files with a CSV parser, then parse these fields with a JSON parser; do not split lines on commas.

History and time split

- A history contains previous **clicks** only (not all earlier impressions): the latest **20 at most**, strictly before the current event, ordered **oldest to newest**.
- `history_item_ids` and `history_timestamps` are aligned element by element and have the same length. An empty history is `[]`. The same item can appear several times.
- The simulated timeline starts before the training period (a *warm-up* period, before 2025-01-20 UTC). Its impression rows are **not** included, but its clicks can appear in histories. A user can have a non-empty history without any row in `train.csv`.
- **Training** covers events from 2025-01-20 00:00 UTC up to, not including, 2025-01-27 00:00 UTC (the cutoff).
- **Evaluation** contains, for each user, the **first event at or after the cutoff**, with all its 10 candidates. Later events of these users are not included.
- Some evaluation users do not appear in `train.csv`, and some evaluation events have an empty history. Make sure your predictions cover these cases too.

Local validation. You have no evaluation labels, so validate on a later part of the training period: fit on earlier events, test on later ones, and keep all rows of an event on the same side of the split. Compute features and aggregate statistics (for example click rates per user or item) using labels from the fitting part only, never from the events you are testing

on. A later training window does not exactly reproduce the official setup, which uses one first event per user.

Submission format

A UTF-8 CSV file with exactly this header (case-sensitive, this order):

```
rowid,predicted_probability
```

Format illustration only – a real submission has one line for **every** evaluation row:

```
rowid,predicted_probability
row_aaaa0000,0.0629
row_bbbb1111,0.1200
row_cccc2222,0.0003
```

Requirements:

- Every `rowid` of `evaluation.csv` appears **exactly once**: no missing, duplicate or unknown rowids.
- Exactly the two columns above and no extra index column. Keep each `rowid` exactly as given (no trimming or case changes).
- Each prediction is a finite number in **[0, 1]**. Plain decimals (`0.25` , `.5` , `1`) and scientific notation (`2e-3`) are accepted. Empty values, text, `nan` and `inf` are rejected.
- Predictions are matched to rows **by** `rowid` , so row order does not matter. Keeping the order of `example_submission.csv` is still convenient.
- The probabilities of an event do **not** have to sum to one.

Upload the finished file through the competition's submission page, then check its processing status in your submission history. If an upload is interrupted, check the history before uploading again.

Evaluation

Submissions are ranked by **ROC AUC**; higher is better. AUC measures how well your predictions rank clicked rows above non-clicked rows: it is the probability that a randomly chosen clicked row gets a higher prediction than a randomly chosen non-clicked row (ties count as one half). Random ranking has an expected AUC of 0.5 and perfect separation gives 1.0. No threshold is needed, and only the ranking matters, not calibration. The example submission is constant and therefore scores ~0.5.

The evaluation rows are split between a public leaderboard and private leaderboard set randomly. Submission scoring computes the ROC AUC for both, but only the public leaderboard score will get updated during Phase 1 of the competition. The final ranking will be determined by the private AUC, to be revealed only after the end of Phase 1. ROC

AUC is computed over all rows in the public/private set rows, and is not a per-user or per-event average.

Each team is limited to 10 submissions per day!

Starter solution

A simple starter solution is provided in starter_solution.py.

To run it yourself, use the command

```
python3 starter_solution.py --train train.csv --evaluation
evaluation.csv --out submission.csv
```

Checklist

- [ ] Read the CSV files with a CSV parser and parse the history columns as JSON.
- [ ] Keep identifiers as text and keep each prediction attached to its `rowid`.
- [ ] Produce one prediction for every evaluation row.
- [ ] Check the header, the number of rows, duplicates, missing or unknown `rowid`s, and that all values are in [0, 1].
- [ ] Check your submission status before retrying an upload.

Setup (i use R for EDA, JupyterLab to run the code)

In [4]:
```r
install.packages(c("data.table", "dplyr", "ggplot2", "stringr"))
library(data.table)   # fast CSV reading
library(dplyr)        #  data wrangling
library(ggplot2)       # plotting
library(stringr)       # for counting items in the history column

# Start with a sample for speed while you build/test your code.
# Once everything works, rerun on the full file by removing nrows=.
train <- fread("train.csv", nrows = 2000000)
```

```
also installing the dependencies 'pkgconfig', 'cpp11', 'farver', 'labeling', 'RCo
lorBrewer', 'viridisLite', 'generics', 'magrittr', 'R6', 'tibble', 'tidyselect',
'gtable', 'isoband', 'S7', 'scales', 'withr', 'stringi'
```

```
package 'pkgconfig' successfully unpacked and MD5 sums checked
package 'cpp11' successfully unpacked and MD5 sums checked
package 'farver' successfully unpacked and MD5 sums checked
package 'labeling' successfully unpacked and MD5 sums checked
package 'RColorBrewer' successfully unpacked and MD5 sums checked
package 'viridisLite' successfully unpacked and MD5 sums checked
package 'generics' successfully unpacked and MD5 sums checked
package 'magrittr' successfully unpacked and MD5 sums checked
package 'R6' successfully unpacked and MD5 sums checked
package 'tibble' successfully unpacked and MD5 sums checked
package 'tidyselect' successfully unpacked and MD5 sums checked
package 'gtable' successfully unpacked and MD5 sums checked
package 'isoband' successfully unpacked and MD5 sums checked
package 'S7' successfully unpacked and MD5 sums checked
package 'scales' successfully unpacked and MD5 sums checked
package 'withr' successfully unpacked and MD5 sums checked
package 'stringi' successfully unpacked and MD5 sums checked
package 'data.table' successfully unpacked and MD5 sums checked
package 'dplyr' successfully unpacked and MD5 sums checked
package 'ggplot2' successfully unpacked and MD5 sums checked
package 'stringr' successfully unpacked and MD5 sums checked

The downloaded binary packages are in
        C:\Users\belgr\AppData\Local\Temp\Rtmp6P9o8I\downloaded_packages
```

```
Attaching package: 'data.table'


The following object is masked from 'package:base':

    %notin%



Attaching package: 'dplyr'


The following objects are masked from 'package:data.table':

    between, first, last


The following objects are masked from 'package:stats':

    filter, lag


The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

1 - Basic shape check

Why: confirms the file loaded correctly and matches what the instructions described, before trusting anything else.

In [5]:
```r
dim(train)              # how many rows, columns
str(train)               # column types
```

```
colSums(is.na(train))   # any missing values per column
```

2000000 · 10

```
Classes 'data.table' and 'data.frame':  2000000 obs. of  10 variables:
 $ rowid             : chr  "row_5e3f316bf724083745def2ed54ffe2db" "row_02f5ad764
538132df510453366f9a522" "row_3f2e9b4601296939dfd9f59819537a97" "row_7e4ea9908c09
7629ba1d430151b5cb92" ...
 $ event_id          : chr  "evt_c8924233d50f30f08baaba91619fd560" "evt_c8924233d
50f30f08baaba91619fd560" "evt_c8924233d50f30f08baaba91619fd560" "evt_c8924233d50f
30f08baaba91619fd560" ...
 $ user_id           : chr  "usr_a5e436d47673e294137606646dc81d15" "usr_a5e436d47
673e294137606646dc81d15" "usr_a5e436d47673e294137606646dc81d15" "usr_a5e436d47673
e294137606646dc81d15" ...
 $ item_id           : chr  "itm_84afdab03779ab1fc1dcb0735bd655a5" "itm_f6dc2fc5c
6c2b00caec556b3779f69a9" "itm_1a6dc0c6d1c02be4c843063dbab74f0c" "itm_809822551f06
39ca8521c81c21fdb095" ...
 $ timestamp         : int  1737331200 1737331200 1737331200 1737331200 173733120
0 1737331200 1737331200 1737331200 1737331200 1737331200 ...
 $ time_bucket       : int  0 0 0 0 0 0 0 0 0 0 ...
 $ day_type          : int  0 0 0 0 0 0 0 0 0 0 ...
 $ history_item_ids  : chr  "[\"\"itm_de0de8f3825d609a883730a6ea449da3\"\",\"\"it
m_d15678985c422ba3a67b91c4230016ac\"\",\"\"itm_4896e9cdd988"| __truncated__ "
[\"\"itm_de0de8f3825d609a883730a6ea449da3\"\",\"\"itm_d15678985c422ba3a67b91c4230
016ac\"\",\"\"itm_4896e9cdd988"| __truncated__ "[\"\"itm_de0de8f3825d609a883730a6
ea449da3\"\",\"\"itm_d15678985c422ba3a67b91c4230016ac\"\",\"\"itm_4896e9cdd988"|
__truncated__ "[\"\"itm_de0de8f3825d609a883730a6ea449da3\"\",\"\"itm_d15678985c42
2ba3a67b91c4230016ac\"\",\"\"itm_4896e9cdd988"| __truncated__ ...
 $ history_timestamps: chr  "[1736868540,1736871180,1736904900,1737011460,1737028
680,1737049200,1737060660,1737065940,1737068940,1737069840,"| __truncated__ "[173
6868540,1736871180,1736904900,1737011460,1737028680,1737049200,1737060660,1737065
940,1737068940,1737069840,"| __truncated__ "[1736868540,1736871180,1736904900,173
7011460,1737028680,1737049200,1737060660,1737065940,1737068940,1737069840,"| __tr
uncated__ "[1736868540,1736871180,1736904900,1737011460,1737028680,1737049200,173
7060660,1737065940,1737068940,1737069840,"| __truncated__ ...
 $ clicked           : int  1 0 0 0 0 0 0 0 0 0 ...
 - attr(*, ".internal.selfref")=<pointer: 0x000001bef8150060>
```

rowid: 0 **event_id:** 0 **user_id:** 0 **item_id:** 0 **timestamp:** 0 **time_bucket:** 0 **day_type:** 0 **history_item_ids:** 0 **history_timestamps:** 0 **clicked:** 0

2,000,000 rows × 10 columns, zero missing values anywhere

2 - Overall click rate

Why: confirms the ~1-in-10 click rate the instructions mention, and is the baseline number everything else gets compared against.

In [6]:
```
train %>%
  summarise(click_rate = mean(clicked))
```

A data.frame: 1 × 1

click_rate
<dbl>
0.062965

0.062965 -> about 6.3% of rows are clicks

Makes sense given the data: each event has 10 candidates but a lot of events have zero clicks at all

3 - Click rate by time of day and weekday vs weekend

Why: checks if timing is a useful signal (e.g. do people click more in the evening, or on weekends?). If yes -> use **time_bucket/day_type** as model features.

In [7]:
```
train %>%
  group_by(time_bucket) %>%
  summarise(click_rate = mean(clicked), n = n())

train %>%
  group_by(day_type) %>%
  summarise(click_rate = mean(clicked), n = n())

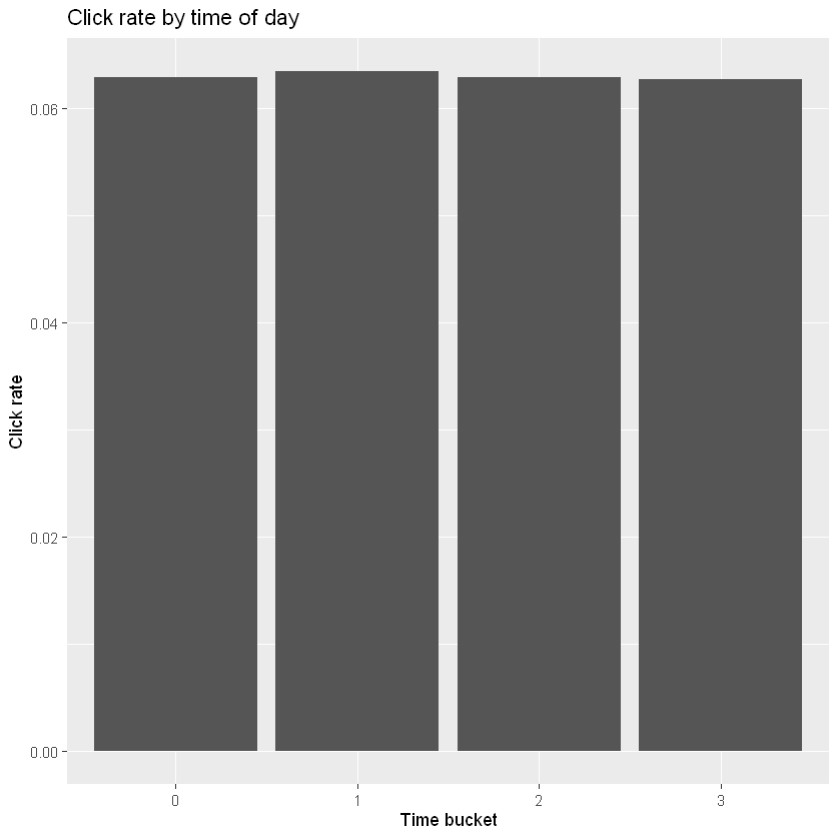
# visualize it
train %>%
  group_by(time_bucket) %>%
  summarise(click_rate = mean(clicked)) %>%
  ggplot(aes(x = factor(time_bucket), y = click_rate)) +
  geom_col() +
  labs(x = "Time bucket", y = "Click rate", title = "Click rate by time of day")
```

A tibble: 4 × 3

time_bucket	click_rate	n
<int>	<dbl>	<int>
0	0.06299003	225750
1	0.06346569	393630
2	0.06289936	592200
3	0.06275716	788420

A tibble: 1 × 3

day_type	click_rate	n
<int>	<dbl>	<int>
0	0.062965	2000000



Time buckets 0–3 all sit at almost exactly the same click rate (~6.28–6.35%) — basically flat, no meaningful time-of-day effect in this sample.

To get weekend rows, grab a chunk from later in the file instead, using skip =:

In [17]:
```
train_tail <- fread("train.csv", skip = 5500000, nrows = 2000000,
                    col.names = c("rowid", "event_id", "user_id", "item_id",
                                  "timestamp", "time_bucket", "day_type",
                                  "history_item_ids", "history_timestamps", "c
```

Then check what day types are in this new chunk:

In [18]:
```
train_tail %>%
  group_by(day_type) %>%
  summarise(click_rate = mean(clicked), n = n())
```

A tibble: 1 × 3

day_type	click_rate	n
<int>	<dbl>	<int>
1	0.06286831	1368241

Compare weekday vs weekend click rates — either on this tail chunk alone, or by combining both:

In [19]:
```
train_combined <- bind_rows(train, train_tail)

train_combined %>%
  group_by(day_type) %>%
  summarise(click_rate = mean(clicked), n = n())
```

A tibble: 2 × 3

day_type	click_rate	n
<int>	<dbl>	<int>
0	0.06296500	2000000
1	0.06286831	1368241

- The two click rates are basically identical - weekday 6.297% vs weekend 6.287%, a gap of just 0.01 percentage points
- That's not a real difference, it's noise-level - statistically day of week doesn't meaningfully affect click rate in this dataset

Conclusion is: time-based features (*time_bucket, day_type*) **don't show a strong standalone signal here**. Users click at roughly **the same rate** no matter what *time* of day or day of *week* it is.

...don't expect time_bucket/day_type alone to meaningfully boost the model so time spent engineering around those two columns specifically probably won't pay off much on its own

4 - History length distribution + the cold-start subset

Why: We need to know how many users have little/no history and whether the model behaves worse for them

In [9]:
```
# count how many items are in each row's history, without fully parsing the JSON

train <- train %>%
  mutate(history_length = ifelse(
    history_item_ids == "[]", 0,
    str_count(history_item_ids, ",") + 1
  ))

# distribution of history length
train %>%
  summarise(
    min_len = min(history_length),
    median_len = median(history_length),
    max_len = max(history_length),
    pct_zero_history = mean(history_length == 0)
  )

ggplot(train, aes(x = history_length)) +
  geom_histogram(binwidth = 1) +
  labs(title = "Distribution of history length per row")

# click rate for low-history vs high-history users
train %>%
  mutate(history_group = case_when(
    history_length == 0 ~ "no history",
    history_length <= 5 ~ "short history (1-5)",
```

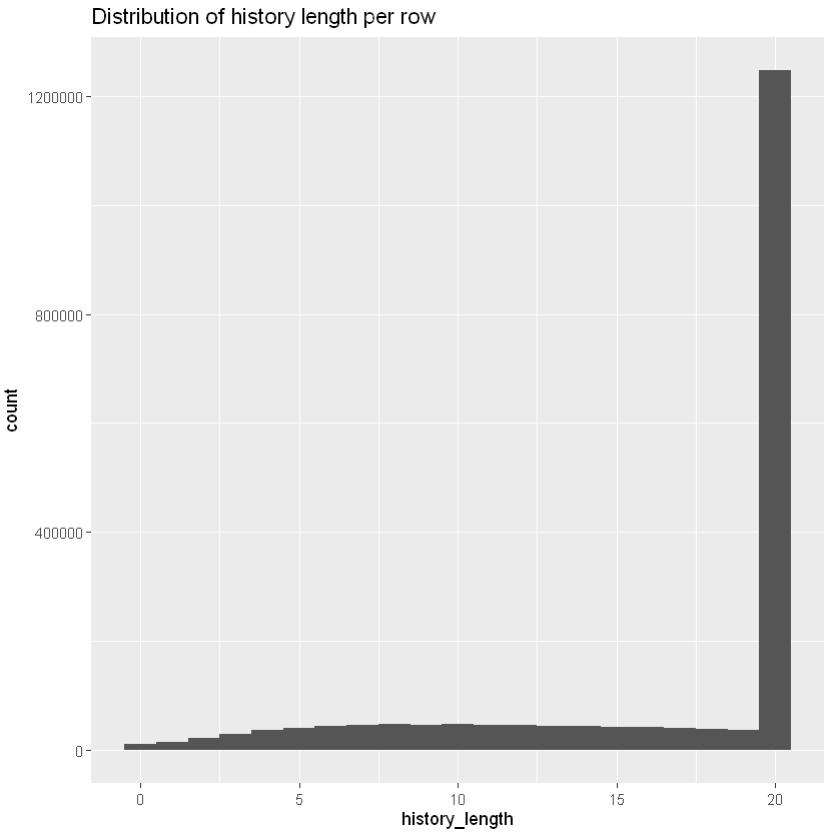
```
    TRUE ~ "long history (6+)"
  )) %>%
  group_by(history_group) %>%
  summarise(click_rate = mean(clicked), n = n())
```

A data.frame: 1 × 4

min_len <dbl>	median_len <dbl>	max_len <dbl>	pct_zero_history <dbl>
0	20	20	0.00469

A tibble: 3 × 3

history_group <chr>	click_rate <dbl>	n <int>
long history (6+)	0.06337849	1849760
no history	0.04349680	9380
short history (1-5)	0.05883146	140860



Most rows have a full 20-item history (median = max = 20) - the histogram confirms it, huge spike at 20. Only 0.47% of rows have zero history (pct_zero_history). The key number: click rate by group -

- No history: 4.35%
- Short history (1-5): 5.88%
- Long history (6+): 6.34%

This is a real, meaningful pattern: users with no history click almost 2 percentage points less often than users with rich history - roughly a 31% relative drop. This is exactly

the **cold-start** insight. It tells: the model **can't rely on the co-click signal for these users** (there's no history to compare against), and they behave measurably differently, so they need to be checked **separately** when judging model performance - not just averaged in with everyone else.

5 - Item popularity skew (popularity bias check)

Why: tells us if a small number of items dominate clicks - relevant to the "popularity bias" idea from the TIGER model, and tells whether popularity alone would already be a strong baseline signal.

In [10]:
```
item_stats <- train %>%
  group_by(item_id) %>%
  summarise(impressions = n(), clicks = sum(clicked), ctr = clicks / impressions

# how skewed is impression count across items?
summary(item_stats$impressions)

ggplot(item_stats, aes(x = impressions)) +
  geom_histogram() +
  scale_x_log10() +
  labs(title = "Item impression count (log scale) — checks for popularity skew")

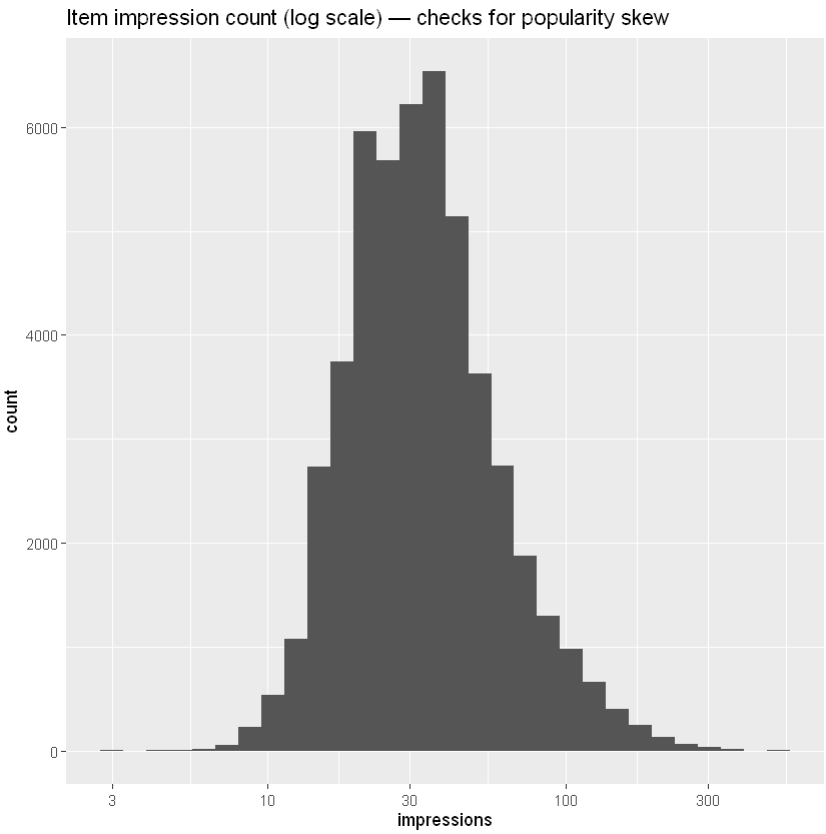
# top 10 most-shown items and their click rate
item_stats %>% arrange(desc(impressions)) %>% head(10)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max. 
      3      22      31      40      47     517 
```

```
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```

A tibble: 10 × 4

item_id	impressions	clicks	ctr
<chr>	<int>	<int>	<dbl>
itm_0acd828c92a74a7ee143759352824672	517	32	0.06189555
itm_0ff61f43cdd85c922e8cc0cd31b013d6	482	37	0.07676349
itm_e98404ec8c3cced09d4d266b8473cf2e	482	43	0.08921162
itm_dbaf615eadf1c8f8c834e017a2d32101	478	33	0.06903766
itm_e9e748324503ef38db2f85629d70b593	478	26	0.05439331
itm_b70402f8a976f622c7945f64fc18ae8c	386	32	0.08290155
itm_ceb82c3339c4d07cc8e5b1d88fb77405	381	30	0.07874016
itm_543d06aca6e90e33f0145e5ab7ae5773	359	32	0.08913649
itm_a24215d44377d7966657986f18217e2d	358	10	0.02793296
itm_a68e86e7b02b4fd8d022011297a60390	355	26	0.07323944



Impressions per item: min 3, median 31, max 517.

- Item popularity is spread fairly smoothly, not dominated by a tiny handful of mega-popular items.
- One extra thing worth noticing in top-10 table: itm_a24215... was shown 358 times but only clicked 10 times (2.8% CTR) - much lower than the others around it (which sit at 6-9%).
  - That's a useful reminder: being shown a lot ≠ being liked a lot - popularity and click-worthiness aren't the same thing, which is exactly why the starter script tracks *item_ctr* separately from raw impression counts.

6 - User activity distribution

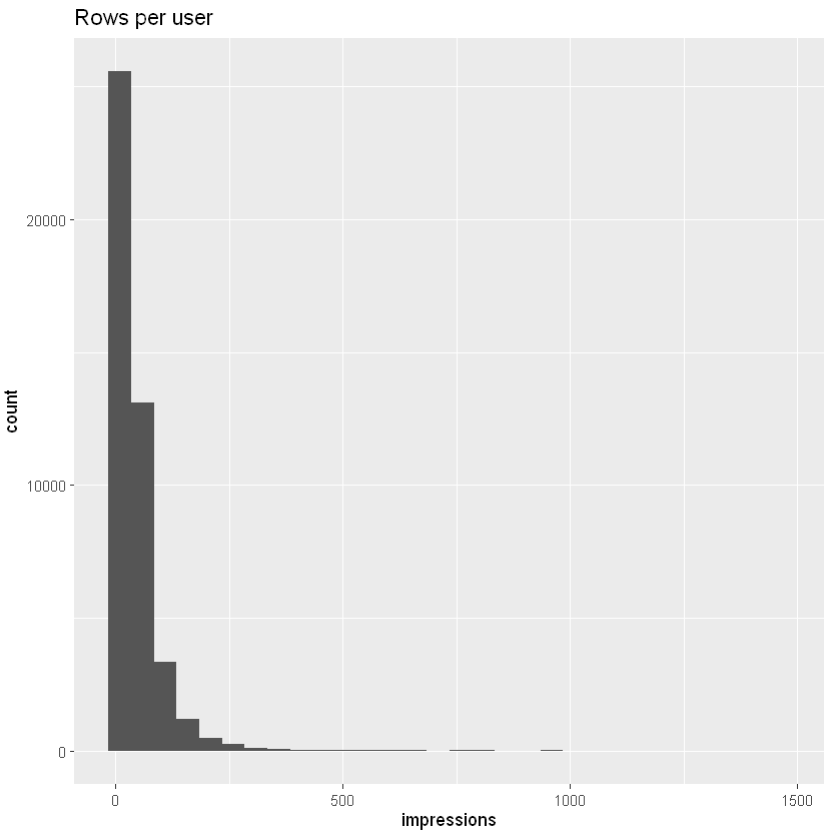
Why: similar idea, but per user - are most users seen only once or twice (hard to personalize for), or do some users have lots of data?

In [11]:
```
user_stats <- train %>%
  group_by(user_id) %>%
  summarise(impressions = n(), clicks = sum(clicked), ctr = clicks / impressions

summary(user_stats$impressions)

ggplot(user_stats, aes(x = impressions)) +
  geom_histogram() +
  labs(title = "Rows per user")
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max. 
  10.00   20.00   30.00   45.23   50.00 1460.00 
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```



Most users have modest activity (10-50 rows), but a small number of users go all the way up to 1460, dragging the average up.

The histogram confirms it - sharp peak near the low end, long thin tail.

Minimum of exactly 10 makes sense too: 1 event = 10 candidate rows, so every user shows up in at least one full event.

7 - Data integrity sanity check

Why: confirms the data actually matches the instructions (10 rows per event, at most 1 click) - catches bugs early

In [12]:
```
event_check <- train %>%
  group_by(event_id) %>%
  summarise(rows_in_event = n(), clicks_in_event = sum(clicked))

table(event_check$rows_in_event)     # should basically all be 10
table(event_check$clicks_in_event)   # should basically all be 0 or 1
```

```
    10
200000

     0      1
 74070 125930
```

- every single one of the 200,000 events has exactly 10 rows
- 74,070 events (37%) had zero clicks, 125,930 (63%) had exactly one click, and critically - no events show 2+ clicks, confirming the "at most one click per event" rule is genuinely true in the data